

Team Text - Software Architecture Overview

Introduction

This document presents the wider architecture developed by Team Text at the KNAW Humanities Cluster. All in-house software mentioned in this documentation can be found via <https://tools.huc.knaw.nl>.

The document serves both as an internal reference, as well as a technical showcase to external parties.

Design Philosophy

- **Reproducibility** - Use the original sources and allow going from zero to full end-results without human intervention. Also support live/incremental updates.
- **Flexibility** - Flexibly accommodate heterogeneous data sources; upstream projects use different formats and different vocabularies.
- **Uniformisation** - Data from distinct sources and projects is transformed into a more uniform vocabulary that our tools understand
- **Service-oriented** - A multi-component service oriented architecture with a web-based user frontend.
- **Scalability** - Handle both small corpora as well as large ones, allow running on a local system (e.g. in development) as well as distributed on a kubernetes cluster for production scenarios.
- **Modularity & Interoperability** - Distinct interoperable software components with clearly distinct functions, no one-serve-all monoliths. Components may be interchangeable for others where appropriate.
- **Reusability** - Components are reusable in other contexts; existing components (incl 3rd party components) are (re)used where applicable.
- **Simplicity** - Though the resulting pipeline is complex, the individual components and their connections retain a certain simplicity. Complexity and abstraction should only be introduced for a good reason.

Service Oriented Architecture and Data pipeline for Text Collections

We have ample experience publishing diverse scientific text collections. These may be literary text editions, historical manuscripts, linguistically-annotated collections or large corpora from automatic OCR or Handwritten Text Recognition.

A data processing pipeline takes the original corpus in the form prepared by the editors, this may be TEI XML, PageXML, FoLiA XML or other formats.

We distinguish the following stages in our pipeline, they are schematically shown below:

1. **Validation** - Validated the corpus as delivered by the editors.

2. **Conversion** - Converts the corpus to plain text and stand-off W3C Web Annotations (radical standoff, via STAM)
 - Standardised linked-open-data vocabulary
 - Text Normalisation (e.g. hyphenation), *support for multiple text representations*
 - Apparatus Conversion
 - IIIF Manifest Generation
3. **Ingest** - Loads the generated artifacts (plain text, annotations, images) into their respective services

The pipeline itself is driven by GNU Make and can be automatically invoked from CI/CD pipelines.

Legend

- Data is shown in yellow
- Processing software is shown in green squares
- Software Services are shown in purple parallelograms
- Communication protocols are shown as arrow labels
- Lines represent the data/provisioning flow
- Software marked with an asterisk is third-party software, all others are developed in-house.
- Tools in the data pipeline are mentioned by their command-line invocations, if they are packaged as a part of a larger suite of tools, that name is between square brackets
- The make targets are given where applicable

Data Models

Data models can be found elsewhere as well (direct links to schemas or READMEs/documentation with schemas):

- STAM
- Text & Annotation Vocabulary

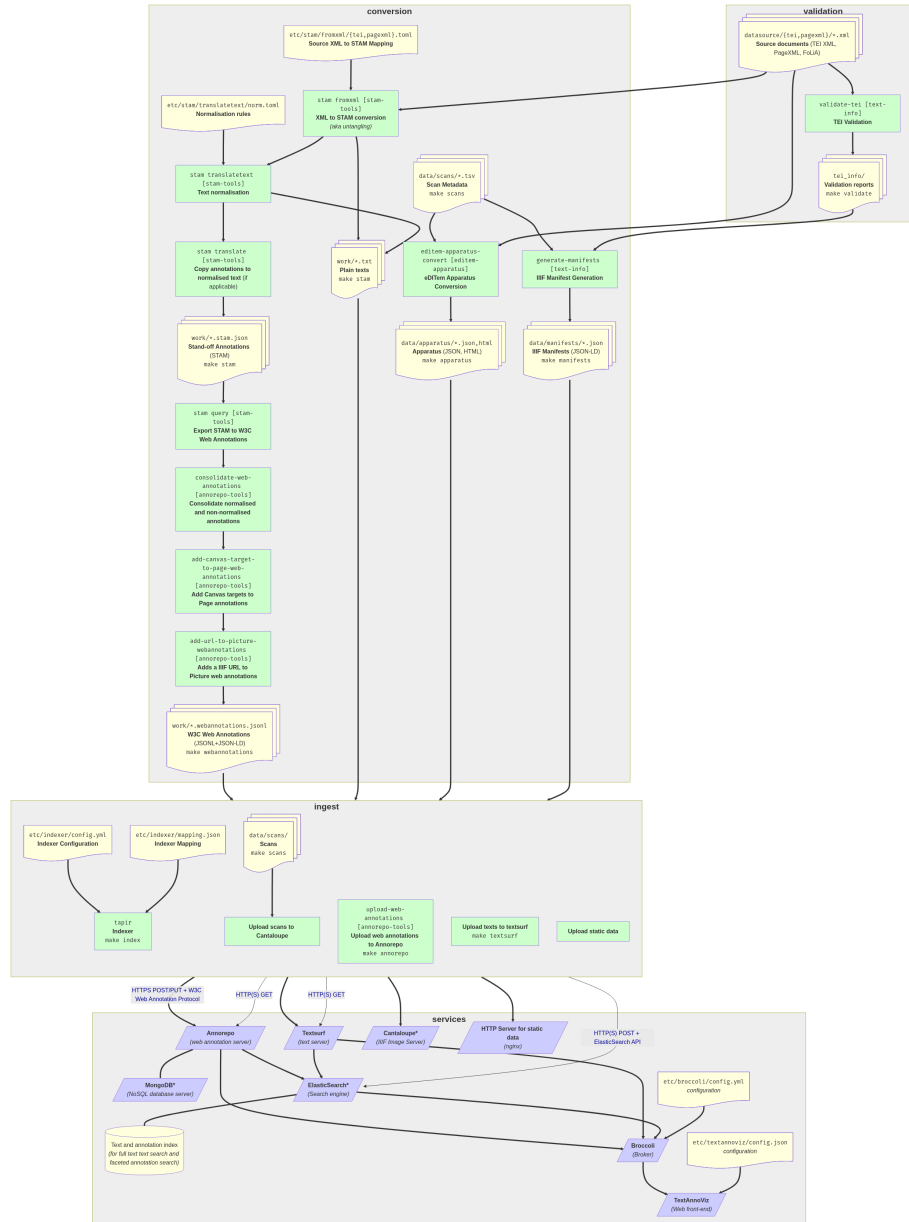


Figure 1: Data Processing and Service Architecture